

Julie BOUTELET
Laurine CHARBONNIER
Quentin GEORGES
Kinjal KAPADIA

Ingénieur en IA
Promotion novembre 2024

Projet TrustPilot

Etape 2 : Data Science

Rendu 1 : Rapport d'exploration, de data visualisation et de pré-processing des données

TABLE DES MATIERES

TABLE DES MATIERES	3
CONTEXTE	4
Contexte d’insertion du projet dans le métier	4
Du point de vue technique	4
Du point de vue économique	4
Du point de vue scientifique	4
OBJECTIFS	6
Objectifs principaux	6
Expertise	6
Projets similaires	6
CADRE	7
Jeu de données	7
Volumétrie	7
Granularité	7
PERTINENCE	8
Pertinence des variables	8
Variable cible	8
Particularités du jeu de données	8
Limitations des données	8
PRE-PROCESSING ET FEATURE ENGINEERING	9
Pre-processing	9
Normalisation	10
Réduction de dimension	10
VISUALISATIONS ET STATISTIQUES	11
Analyse générale	11
Analyse textuelle	12

CONTEXTE

Contexte d'insertion du projet dans le métier

Ce projet s'inscrit dans le domaine du **Machine Learning appliqué à l'analyse d'avis clients**.

Notre objectif est de concevoir un **modèle de classification automatique** des avis publiés sur **Trustpilot**, afin d'en extraire des informations utiles.

Concrètement, nous cherchons à identifier automatiquement les thèmes évoqués dans chaque avis (ex: service client, livraison, qualité des produits) pour **renforcer la transparence et la lisibilité** des notes d'entreprises.

Cette catégorisation aiderait les utilisateurs à comprendre dans **quels domaines une entreprise excelle ou rencontre des difficultés**, et permettrait aux entreprises de **cibler plus efficacement leurs axes d'amélioration**.

Du point de vue technique

Le projet repose sur la conception d'un **modèle de classification multi-label**.

Chaque avis textuel est prétraité : nettoyage (si nécessaire en fonction du modèle utilisé avec suppression des stopwords, lemmatisation...), vectorisation (TF-IDF, Transformers...), ensuite, les données n'étant pas labellisées, nous utilisons des **techniques de topic modeling** (BERTopic, LDA...) pour regrouper les avis en thèmes cohérents, avant d'en nommer les clusters à l'aide d'analyses statistiques.

Les données ainsi annotées servent ensuite à entraîner un modèle supervisé de classification.

A terme, le modèle sera intégré dans une pipeline de prédiction automatisée permettant de **catégoriser les nouveaux avis en temps réel et de mettre à jour dynamiquement les notes moyennes par catégorie**.

L'interface utilisateur permettra de visualiser les résultats et tendances par catégorie.

Des métriques comme l'accuracy, la précision, le rappel seront utilisées pour évaluer les performances du modèle.

Du point de vue économique

D'un point de vue économique, ce projet apporte une **valeur ajoutée significative** à Trustpilot.

La plateforme se différencierait de ses concurrents, en proposant une **analyse thématique automatisée** des avis, ce qui renforcerait **la fidélisation des utilisateurs et la confiance dans les notations**.

Pour les entreprises, cette nouvelle fonctionnalité permet de **réduire le temps et les coûts liés à l'analyse des retours clients**.

Pour les consommateurs, elle offre une **lecture plus claire et plus utile des avis**, améliorant ainsi l'expérience utilisateur et donc le trafic sur la plateforme.

Du point de vue scientifique

Scientifiquement, le projet s'appuie sur des **approches avancées de traitement automatique du langage naturel (NLP)**.

Il mobilise des techniques de **topic modeling**, de **vectorisation sémantique** (TF-IDF, BERT embeddings...), et de **classification multi-label**.

L'expérimentation inclut également une **évaluation empirique** des modèles afin d'identifier celui offrant le meilleur compromis entre performance et coût d'entraînement.

Le projet illustre donc la mise en œuvre du **cycle complet d'un projet de machine learning** : de l'exploration et du traitement des données à la mise en production du modèle, en passant par la modélisation et l'évaluation de celui-ci.

OBJECTIFS

Objectifs principaux

Notre objectif est de construire un **modèle de classification** qui prend un avis client (texte) et prédit un ou plusieurs thèmes/étiquettes décrivant le **contenu de cet avis** (par exemple : livraison, produit, service client, prix etc), afin d'automatiser l'analyse qualitative des retours clients. N'ayant **pas de données labellisées** pour entraîner ce modèle de classification, notre premier objectif est d'abord de **créer ces données**, à l'aide de techniques de **topic modeling**.

Expertise

Aucun d'entre nous n'a d'expertise particulière sur la problématique, hormis Laurine, qui possède un master en NLP. Nous avons cependant été en contact avec notre **mentor de projet**. Nous avons mis en place une collaboration continue avec notre mentor de projet, un expert dans le domaine de la Data Science.

Ces échanges nous ont permis d'affiner notre problématique :

- Le choix de labels thématiques pertinents,
- L'adaptation de notre modèle pour une meilleure gestion des avis,
- La sélection des méthodes (embedding, vectorisation) les plus adaptées.
- Il nous a aidés à aligner nos choix sur les priorités métier.
- Le projet est bien fait et utile pour l'entreprise ou l'organisation concernée.

Projets similaires

Notre projet est réalisé dans un **cadre académique** et ne dépend d'aucun projet existant au sein de notre établissement. Toutefois, nous nous sommes inspirés de **projets similaires d'analyse d'avis clients** disponibles en **open-source**, notamment pour le choix des modèles de topic modeling et des modèles d'embeddings.

CADRE

Jeu de données

Le jeu de données est nommé **reviews_trust.csv**. Ce jeu de données nous a été fourni par les **équipes TrustPilot**. Il s'agit de données concernant 2 entreprises "ShowRoom" et "Veepee" avec les commentaires et notes des avis laissés, ainsi que des métadonnées, telles que la date et le nom du client ayant écrit l'avis.

Ces données ne sont pas librement disponibles. Elles appartiennent à l'entité "**TrustPilot**" pour laquelle nous travaillons. Ils sont propriétaires de la donnée.

Volumétrie

Les dimensions du jeu de données sont **19863 lignes** pour **11 variables**. Le fichier pèse **0.3 Mo**. 2 variables sont de type numérique, et le reste (9 variables) de type textuelle. On a en moyenne **5.5 % de valeurs manquantes** sur tout le dataframe réparti en fonction des variables comme cela :

- Commentaire : 0.02 %
- Star : 0.00 %
- Date : 0.28 %
- Client : 7.10 %
- Réponse : 8.38 %
- Source : 0.00 %
- Company : 0.00 %
- Ville : 11.04 %
- Maj : 14.62 %
- Date Commande : 9.70 %
- Ecart : 9.70 %

La valeur numérique "**Star**" doit être **bornée entre 1 et 5**, sinon il s'agit de valeurs aberrantes.

Toutes les valeurs sont bien comprises entre 1 et 5, il n'y a donc pas de valeurs aberrantes.

Pour ce qui est de la variable source, il y a **deux sources**: "*Trusted Shop*" et "*Trust Pilot*": il y a 427 valeurs en doublon. Il serait intéressant de nettoyer le jeu de données par la suite.

Granularité

Ici la granularité représente un commentaire d'un individu posté à un moment t. La période couverte est de **Octobre 2015 à Juin 2021**.

Le jeu est statique mais peut être amené à devenir dynamique avec des données mises à jour régulièrement. Par exemple, possibilité de visualiser les données en temps réel, de manière journalière ou mensuelle.

PERTINENCE

Pertinence des variables

Pour le premier objectif de création du dataset avec le topic modeling, la seule variable pertinente est **Commentaire**, car elle contient nos données textuelles à catégoriser.

Pour l'objectif final de classification multi-label des avis, voici les variables les plus pertinentes :

- **Star** : cette variable numérique représente la note attribuée par le client ; utilisée pour interpréter le sentiment associé à chaque avis (négatif, positif).
- **Commentaire** : le texte de l'avis, utilisé pour le prétraitement linguistique, la lemmatisation et la vectorisation.
- **Date** : il est possible d'analyser comment les avis évoluent avec le temps, en identifiant les tendances et les périodes de forte insatisfaction.
- **Réponse** : pour mesurer l'effort de l'entreprise à interagir, indique si l'entreprise a répondu ou non à l'avis.

Variable cible

L'objectif du projet est **d'identifier et de grouper les avis clients** selon des thématiques ou catégories (Par exemple: livraison, qualité du produit, service client etc). Pour le moment, la **variable cible** n'est pas encore disponible dans le jeu de données initial, car les avis ne sont pas labellisés. L'objectif est donc de créer ces labels (par clustering) afin de construire un jeu de données annoté. Pour cela, nous utilisons des **techniques de topic modeling** (BERTopic, LDA...) pour regrouper les avis en thèmes.

Une fois les données labellisées, nous pourrions envisager une **classification supervisée**, où la variable cible correspondra aux **groupes thématiques identifiés**.

En résumé, pour l'instant, la **variable cible** est à construire. L'objectif est de regrouper les commentaires par thèmes et d'attribuer un label à chaque groupe. Si la qualité des données le permet, une classification pourra ensuite être réalisée à partir de ces labels.

Particularités du jeu de données

Le nombre de **NA** dans certaines colonnes limite leur utilisation. Le jeu de données nécessite aussi un travail de regroupement et d'annotation pour **créer la variable cible**.

Les avis sont en **langue française**, nécessitant l'utilisation d'outils NLP spécifiques (spaCy français, CamemBERT, etc.)

Limitations des données

Le dataset ne contient pas de labels thématiques explicites (livraison, qualité, service, etc.), ce qui rend difficile une classification par sujet. Certaines **valeurs manquantes** limitent l'exploration. Les **avis très courts** ou **sans texte** ne contiennent pas assez d'informations pour être utiles à la modélisation.

PRE-PROCESSING ET FEATURE ENGINEERING

Pre-processing

Comme nous utilisons uniquement la colonne "Commentaire", nous ne nous sommes pas préoccupés des autres colonnes. Nous avons cependant effectué plusieurs traitements sur les commentaires. Nous avons tout d'abord nettoyé la colonne, afin de **supprimer les valeurs vides et les doublons**. Nous avons également supprimé les **commentaires contenant trois mots ou moins**, car nous considérons qu'ils ne sont pas suffisamment longs pour évoquer un thème. Enfin, nous avons effectué un dernier tri sur les commentaires : une minorité étant dans **d'autres langues** que le français, nous les avons supprimés à l'aide du modèle de **détection de langue** Fasttext, auquel nous avons appliqué des seuils de probabilités afin de préciser le filtrage. Après ce premier nettoyage des données, nous passons de 19863 à 15077 commentaires.

Nous avons ensuite appliqué un premier traitement permettant de nettoyer le texte : à l'aide d'**expressions régulières**, nous avons supprimé les potentiels URLs, adresses mail, mentions @ et hashtags, et nous avons remplacé les espaces multiples, sauts de lignes et tabulations par des espaces simples.

Ensuite, comme nous avons testé plusieurs techniques de **vectorisation du texte**, nous n'avons pas appliqué les mêmes traitements selon la technique utilisée.

Nous avons commencé par une approche statistique, le **TF-IDF**. Cette technique permet d'évaluer **l'importance d'un terme dans un document** (ici, un commentaire) par rapport à une collection de documents (ici, l'ensemble des commentaires). Comme il s'agit principalement de compter le nombre d'apparitions des termes, nous avons traité le texte afin de faciliter le comptage. A l'aide de la **librairie Spacy**, nous avons donc **tokenisé** le texte, puis nous avons **lemmatisé** chaque token, et supprimé les **stopwords** ainsi que les tokens non alphanumériques. Nous avons ensuite **vectorisé** le texte à l'aide du TF-IDF, en supprimant les tokens apparaissant dans plus de 95% des documents et dans moins de 5 documents.

Nous avons ensuite voulu tester une méthode de **word embeddings**. Basés sur le deep learning, ceux-ci permettent de représenter des mots sous forme de vecteurs numériques dans un **espace multidimensionnel**, qui peut ensuite être utilisé indépendamment. Le modèle **Word2Vec** est efficace pour préserver la **similarité contextuelle** entre les mots lors de la vectorisation. Nous avons utilisé **FastText**, qui est une version améliorée de Word2Vec, car il permet de prendre en compte des **n-grams de caractères**. Dans notre cas, les commentaires pouvant contenir des fautes d'orthographe, ce modèle semble donc approprié. Pour cette approche, nous avons effectué le même prétraitement que pour le TF-IDF.

Enfin, nous avons testé une solution plus avancée, utilisant les **réseaux de neurones profonds**. Ceux-ci permettent de créer des embeddings de phrases à l'aide de **vecteurs à haute dimension**. Les **Transformers** font partie des techniques les plus utilisées : ceux-ci utilisent des mécanismes d'attention pour capturer les propriétés sémantiques des mots. **BERT** est un modèle basé sur les Transformers, qui produit des vecteurs à

l'aide d'un encoder bi-directionnel. Celui-ci est très général, et peut être fine-tuné pour l'adapter à toutes sortes de tâches. **Sentence-BERT** est un modèle basé sur BERT, qui permet de générer des **représentations sémantiques de phrases** sous forme de vecteurs. Pour créer nos embeddings de phrases, nous avons testé deux versions : Sentence-BERT couplé avec **CamemBERT**, la version française de BERT, ainsi que Sentence-BERT couplé avec "paraphrase-multilingual-MiniLM-L12-v2", un **modèle multilingue** prévu pour les embeddings de phrases. Pour cette méthode de vectorisation, nous n'avons pas appliqué de traitements préalables, car ce type de modèle capte mieux la sémantique avec le texte brut.

Normalisation

Le TF-IDF et Word2Vec produisent des vecteurs dont la **taille dépend du nombre de mots** que contient le texte encodé. Il est donc nécessaire de normaliser les vecteurs obtenus afin d'éviter de biaiser le modèle en donnant plus de poids aux commentaires plus longs. Nous avons donc utilisé la **normalisation L2** afin d'attribuer une **norme égale à 1** à tous les vecteurs obtenus. De plus, certains modèles de type Sentence-BERT produisent des vecteurs déjà normalisés, mais comme ce n'est pas le cas pour tous, nous avons préféré appliquer la normalisation L2 sur ces vecteurs également.

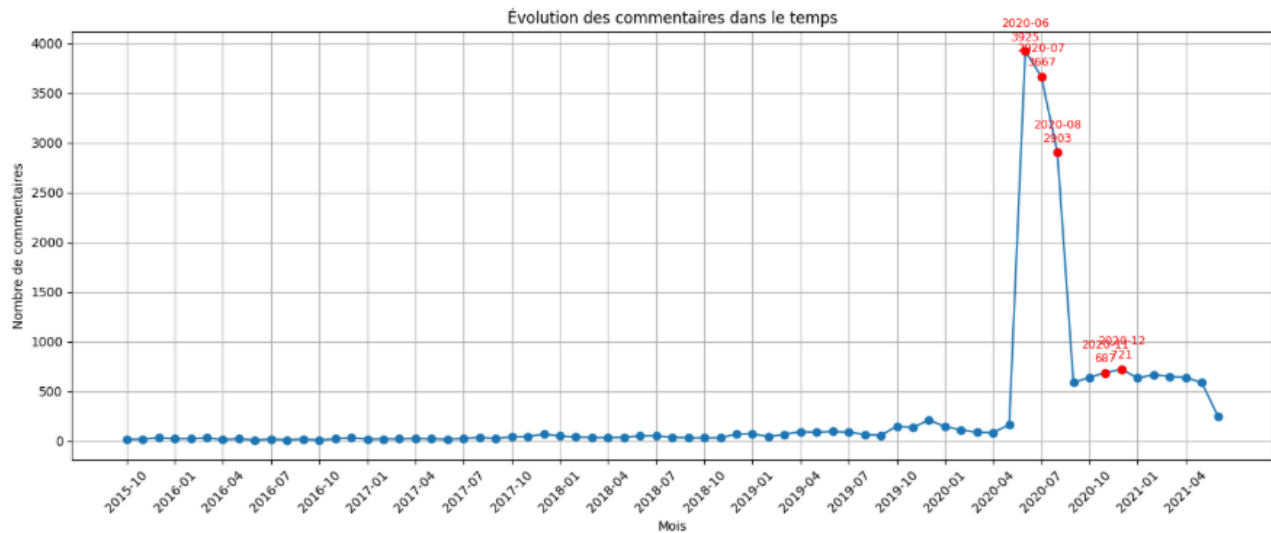
Réduction de dimension

En fonction du **nombre de dimensions recommandé** pour les modèles que nous testerons, la réduction de dimension pourra être envisagée. Les embeddings obtenus avec le TF-IDF seront très probablement réduits, car cette technique crée une dimension très élevée : nous avons dans notre cas **3788 dimensions**. Cela s'explique par le fait qu'une dimension correspond à un **mot unique** présent dans le texte vectorisé. Pour Word2Vec, nous avons choisi une dimension de 300, la réduction ne devrait donc pas être nécessaire. Quant à Sentence-BERT, cela dépendra des besoins du modèle de topic modelling : pour CamemBERT, nous avons 768 dimensions, et avec le modèle multilingue, nous obtenons 384 dimensions.

VISUALISATIONS ET STATISTIQUES

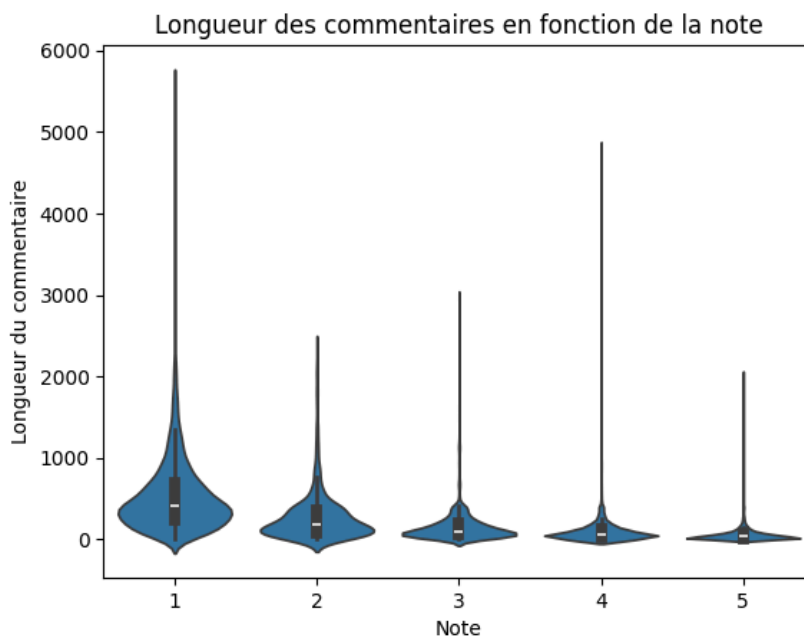
Analyse générale

Nous avons cherché à regarder les variables dans leur globalité avant de se concentrer sur les variables essentiellement textuelles.

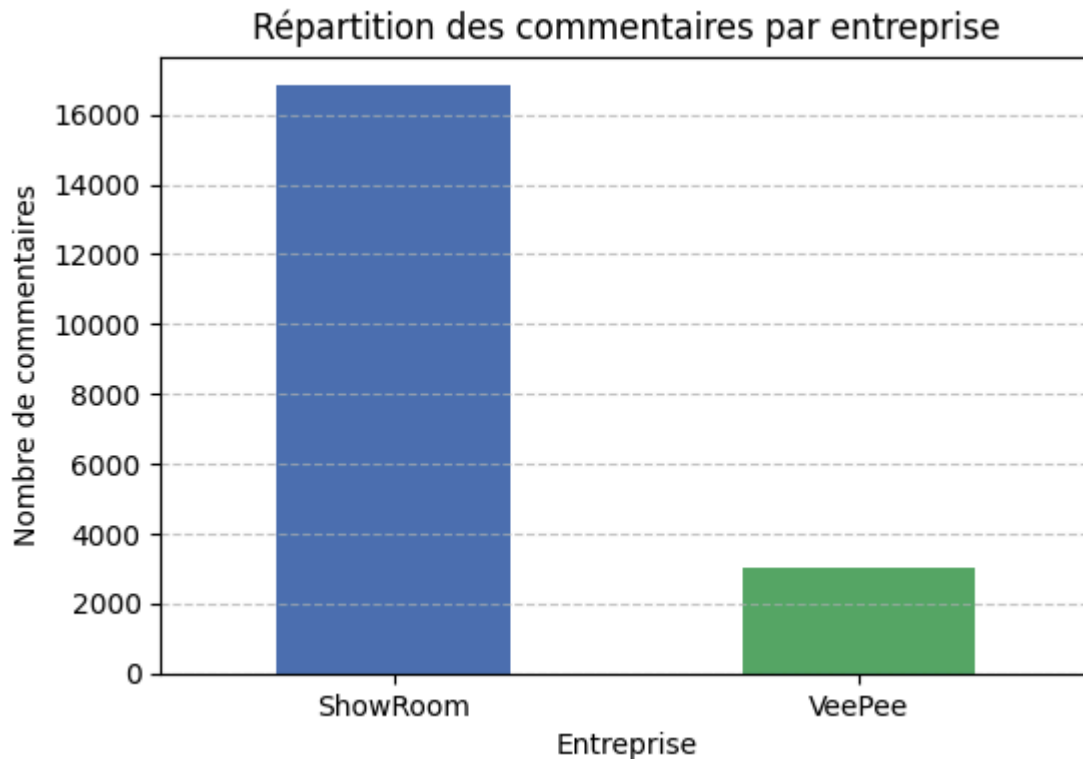


Il y a un **pic de commentaires** en Juillet 2020, avec un **pic à 4000 commentaires** ce qui contraste avec les 100 commentaires moyens par mois.

Nous restons sur un traitement textuel des données pour ce projet, mais si le projet venait à devoir être interprété par mois, il serait intéressant de comprendre ces données “outliers”.



Le **graphique en violon** ci-dessus montre la distribution de la **longueur des commentaires** en fonction de la **note** attribuée par les clients (de 1 à 5). On observe que les avis dont la note est de 1 ou 2 ont tendance à être accompagnés de commentaires plus longs que les avis positifs (4 et 5).



Le graphique ci-dessus montre un **déséquilibre** important dans le dataset : la majorité des commentaires proviennent du site **ShowRoom** (environ 84%), tandis que **VeePee** représente environ 16% des données.

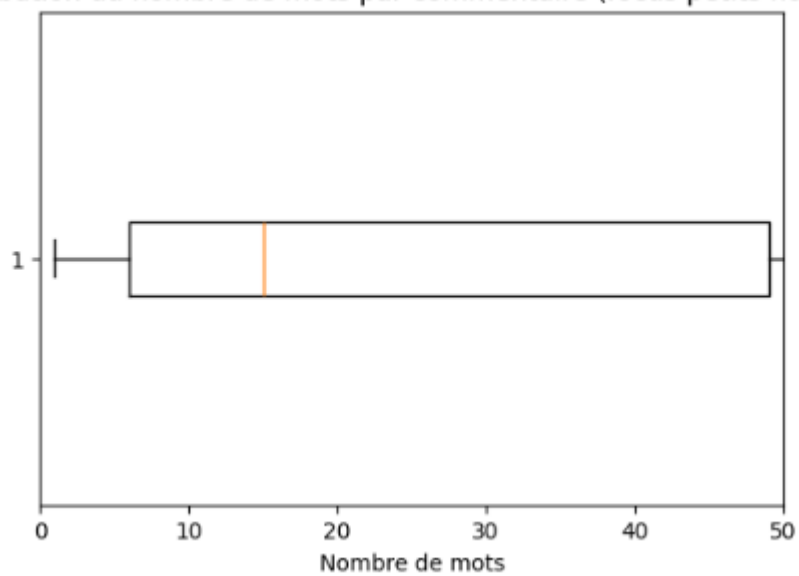
Cependant, comme ces deux entreprises opèrent dans le **même secteur d'activité**, et partagent donc les mêmes processus de vente/achat, nous pouvons exploiter l'ensemble du dataset sans distinction des deux sources.

Analyse textuelle

Concernant la distribution des mots, nous remarquons que la majorité (entre le premier quartile et le troisième quartile) des commentaires ont entre **6 et 49 mots** :

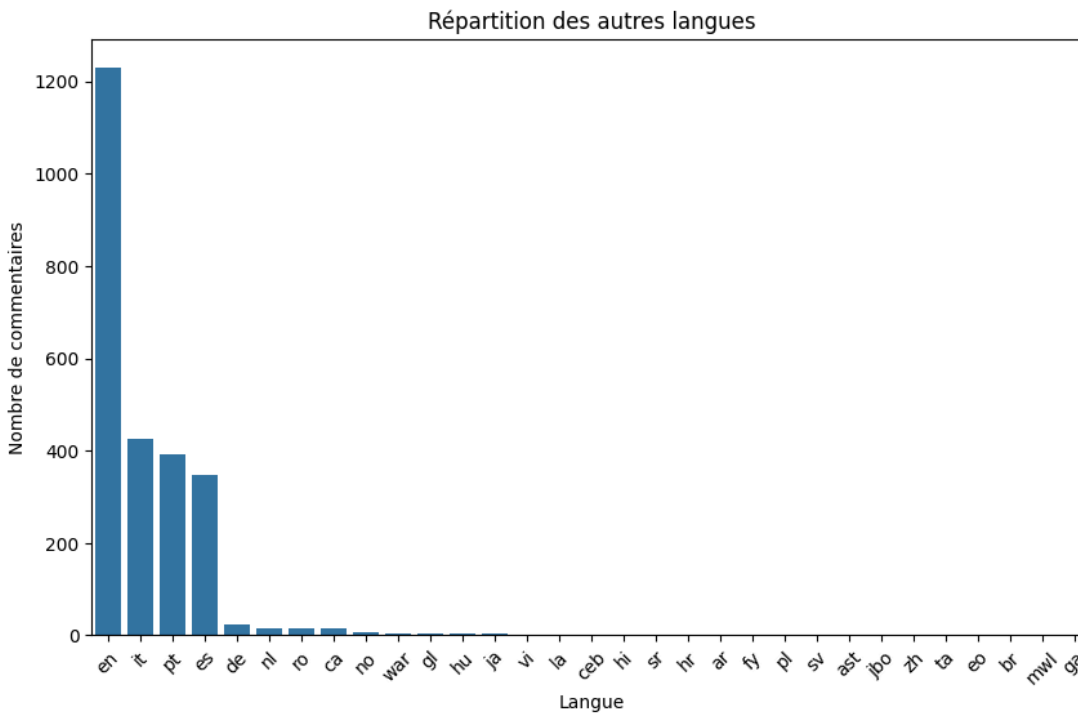
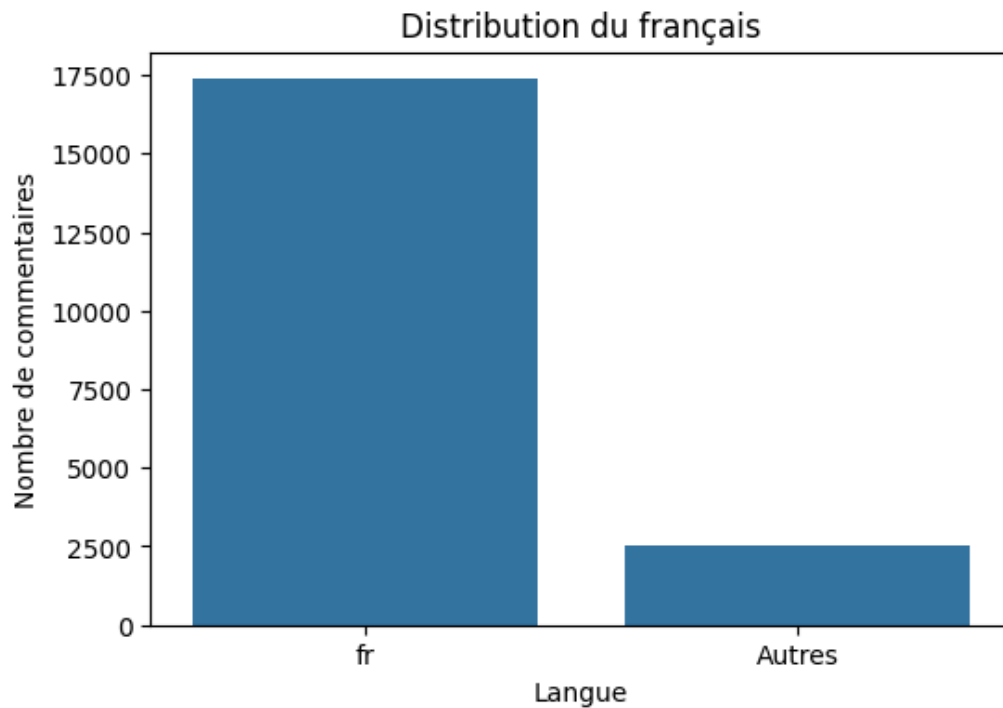
```
: count    19863.000000
  mean      40.624528
  std       64.674733
  min        1.000000
  25%        6.000000
  50%       15.000000
  75%       49.000000
  max      1040.000000
```

Distribution du nombre de mots par commentaire (focus petits nombres)



Pour une meilleure interprétabilité du modèle, on choisit un seuil de plus de 3 mots pour un commentaire, afin d'être sûr que le commentaire puisse avoir une thématique que l'on pourrait faire ressortir.

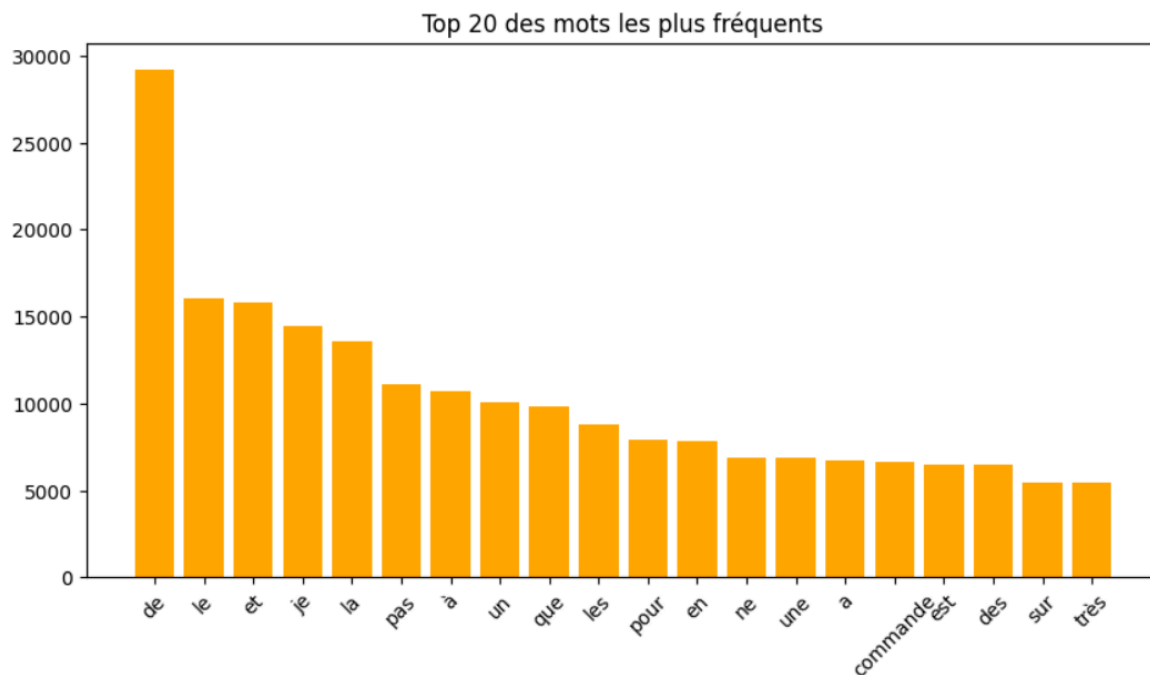
13% des commentaires ont moins de 3 mots, donc ce nettoyage n'entraînera pas une perte de données massive.



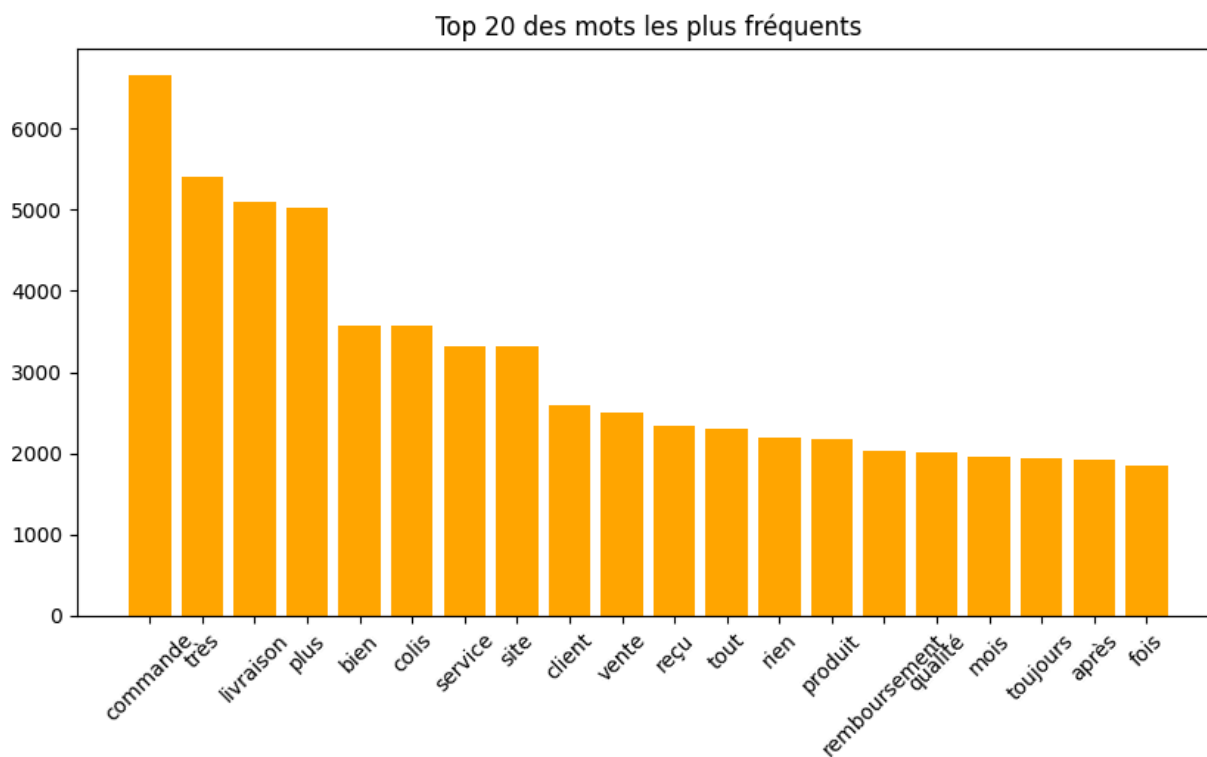
Les deux graphiques ci-dessus montrent la **répartition des langues** dans le dataset. Ces données ont été obtenues à l'aide du **modèle de détection de langues** FastText. Nous n'avons gardé que la langue détectée comme la plus probable par le modèle ; le résultat n'est donc pas toujours correct. Nous avons cependant vérifié les résultats à la main ; la détection du français est **quasi-systématiquement correcte**.

On peut voir que **le français est majoritaire**, c'est pourquoi nous avons décidé de nous concentrer sur cette langue et de supprimer les autres commentaires lors de l'étape de pré-processing. Parmi les autres langues

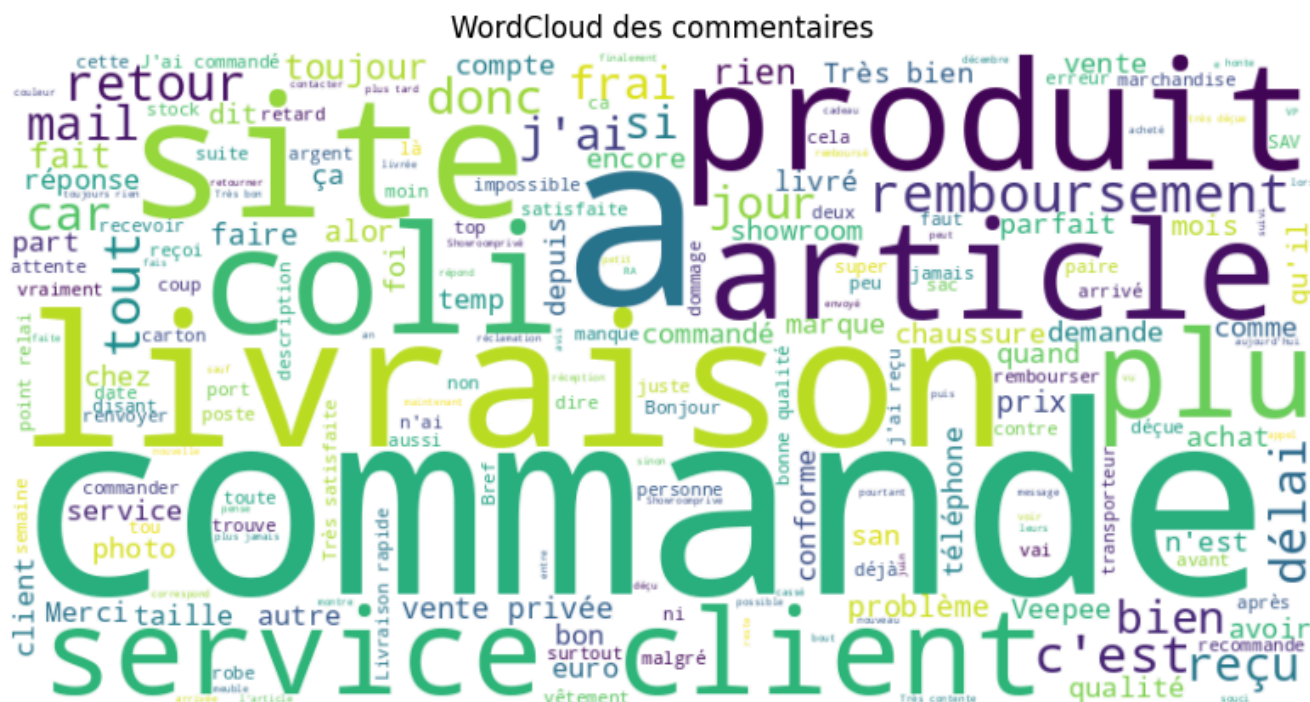
présentes dans le dataset, on trouve principalement l'anglais, suivi de l'italien, le portugais et l'espagnol. Les autres langues présentées sont très minoritaires.



Le graphique montre les **20 mots les plus fréquents** dans les avis après un nettoyage de base. On remarque la forte présence de mots fonctionnels, indiquant la nécessité de **supprimer les stopwords** pour mieux cibler les mots porteurs de sens.

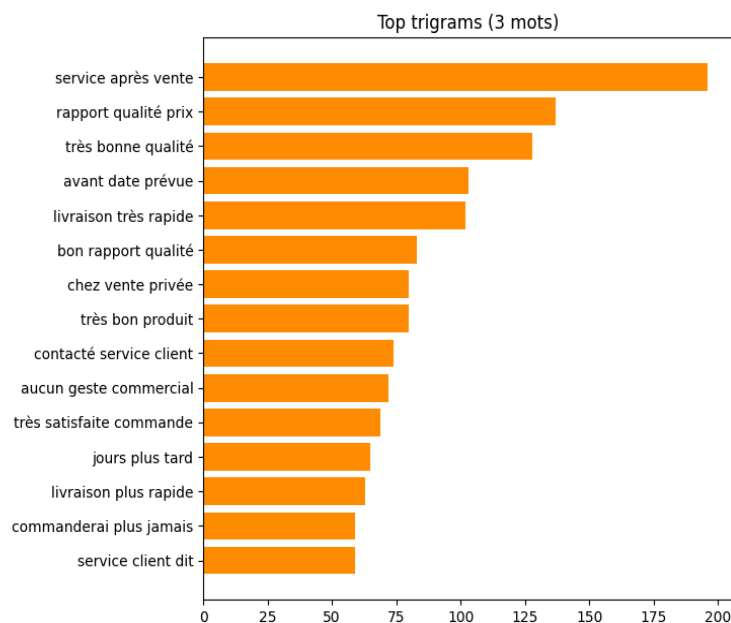
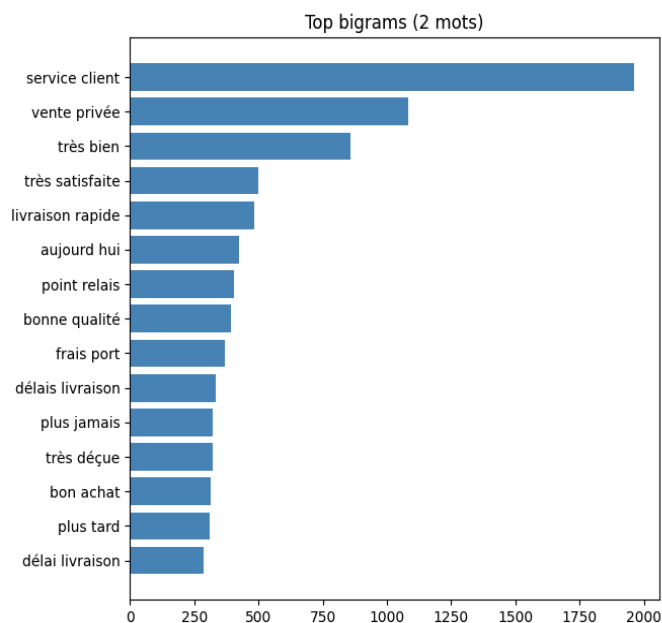


Après **suppression des stopwords**, on commence à voir **apparaître quelques thèmes**, avec le termes "commande", "livraison", "colis", "service", "remboursement", et "qualité".



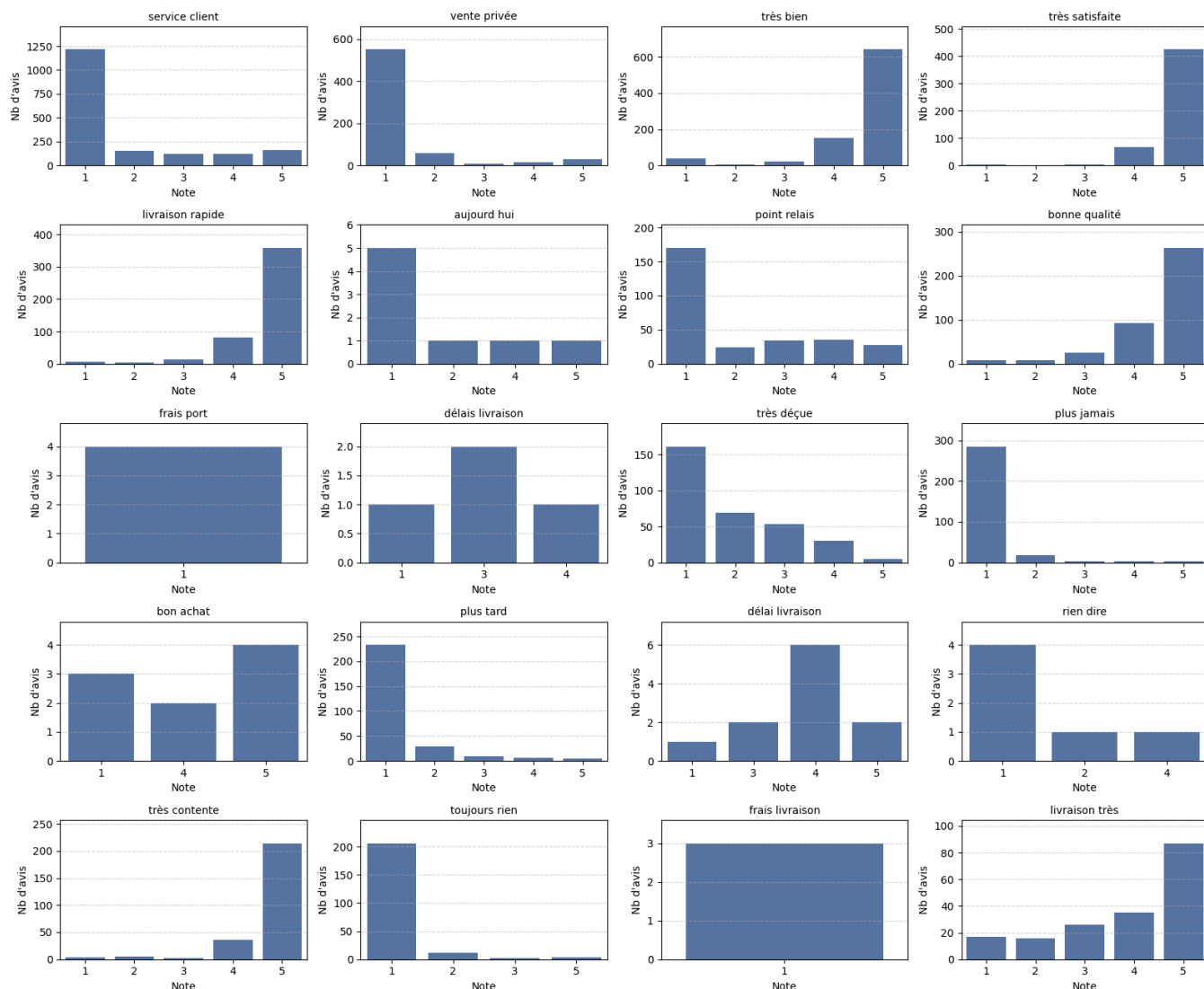
Le wordcloud ci-dessus permet de constater la fréquence des **termes les plus utilisés** dans les commentaires clients. Plus un terme apparaît gros, plus il est fréquent dans notre dataset.

Les mots comme commande, livraison, article, produit, colis, site, service client semblent apparaître le plus souvent indiquant que les retours des utilisateurs se concentrent principalement sur le **processus d'achat** et la **qualité du service**.



Les deux graphiques en barres ci-dessus montrent les 15 **bigrammes** et **trigrammes** les **plus fréquents** dans notre dataset. On y retrouve les thèmes du **service client**, des **livraisons**, et de la **qualité du produit**. Plusieurs n-grams montrent également un **fort avis positif ou négatif**, comme "très satisfaite", "très bon produit", "très bien", mais aussi "très déçue", et "plus jamais". Si le projet venait à s'orienter vers de l'analyse de sentiments, ces termes seraient très utiles et on pourrait pousser cette analyse dans ce sens.

Répartition des notes selon les 20 bigrammes/trigrammes les plus fréquents



Ces graphiques représentent la répartition des notes des avis contenant les 20 bigrammes les plus fréquents dans les commentaires.

Nous observons par exemple que les bigrammes “livraison rapide”, “très bien”, “très satisfaite”, “bonne qualité”, “très contente” sont presque exclusivement associées à des notes de 5.

Tandis que des bigrammes comme “service client”, “plus jamais”, “toujours rien” sont majoritairement associés à des notes de 1 ou 2.

On peut également remarquer que certains bigrammes, bien qu'ils ne semblent pas avoir une connotation positive ou négative, semblent tout de même être associés à des notes spécifiques, surtout dans le cas des

avis négatifs. Par exemple, "service client" et "vente privée", "point relais" et "aujourd'hui" sont plus souvent associés à une note de 1.

Rendu 2 : Rapport de modélisation

Table des matières

Table des matières	2
I. Topic modeling	3
Classification du problème	3
BERTopic	3
CTM (Contextualized Topic Models)	5
TOP2VEC	7
LDA (Latent Dirichlet Allocation)	8
Choix du modèle	10
Interprétation des résultats	10
Annotation manuelle des clusters	11
II. Annotation weakly-supervisée par règles linguistiques	13
III. Classification multi-label	16
Classification du problème	16
XGBoost	16
CamemBERT fine-tuné	20
KNN	23
SVM	25
Régression Logistique	27
Interprétation des résultats	31

I. Topic modeling

Classification du problème

L'objectif du projet étant d'obtenir des données labellisées pour ensuite permettre d'entraîner un modèle de classification, celui-ci s'apparente tout d'abord à du clustering, plus précisément à du topic modeling. Il s'agit d'une tâche d'annotation de commentaires en multilabel. Trois classes ont été définies au cours du projet : qualité produit, service livraison, service client.

Les métriques que nous avons utilisées pour comparer nos modèles de topic modeling sont le score de cohérence et la diversité des topics. Le score de cohérence permet de mesurer à quel point les mots importants d'un même topic apparaissent souvent ensemble dans les documents, tandis que la diversité des topics mesure si les topics ne contiennent pas tous les mêmes mots. Nous avons privilégié le score de cohérence, car le plus important pour nous était que chaque topic soit représentatif d'un thème. Le cas où plusieurs topics concernent le même thème n'était pas un problème, puisqu'il était ensuite question de les regrouper manuellement.

En plus de ces métriques quantitatives, nous avons également fait une évaluation qualitative, en analysant manuellement les clusters obtenus afin d'obtenir une interprétation humaine. Nous avons également tenté de visualiser les clusters sur des graphiques, mais ceux-ci étaient trop nombreux et trop peu qualitatifs pour être interprétables.

Nous avons essayé quatre modèles : LDA, Top2Vec, CTM, et BERTopic. Pour chaque modèle, nous allons présenter son fonctionnement, le prétraitement nécessaire, l'ajustement qui a été effectué, ainsi que les résultats obtenus.

BERTopic

Principe du modèle

Le modèle BERTopic combine des embeddings sémantiques avec des algorithmes de clustering et de réduction de dimension. Contrairement aux méthodes classiques, il ne se base pas uniquement sur la fréquence des mots, mais comprend également le sens des phrases grâce aux modèles de langage. Cela permet que deux textes avec des mots différents mais des sens proches puissent être regroupés dans un même topic.

La première étape dans la Pipeline du modèle consiste donc à créer des embeddings de phrases avec Sentence-BERT. Ensuite, BERTopic utilise UMAP pour projeter les vecteurs dans un espace de plus faible dimension, tout en préservant leur structure sémantique.

Cette étape permet de réduire la dimension des embeddings, afin de faciliter le clustering. Celui-ci est effectué avec l'algorithme HDBSCAN, qui a l'avantage de ne pas nécessiter de fixer un nombre de topics souhaité. Il permet de gérer des clusters de tailles variables et détecte les documents hors-topic (outliers), en

leur assignant le topic -1. La dernière étape de BERTopic consiste à créer une représentation des topics : chaque topic est décrit avec ses mots les plus caractéristiques, à l'aide du class-based TF-IDF, qui considère chaque topic comme un document.

Prétraitement et embeddings

Nous avons effectué le prétraitement cité dans le rapport de preprocessing pour les embeddings de phrases. Trois embeddings différents ont été testés : "paraphrase-multilingual-MiniLM-L12-v2" , "all-mpnet-base-v2", et "camembert-base". Les deux premiers sont des embeddings multilingues, tandis que le dernier est français. La vectorisation a été effectuée avec un CountVectorizer.

Ajustement du modèle

Le modèle a été testé sur les trois embeddings afin de sélectionner celui qui fonctionnait le mieux. L'embedding retenu est "paraphrase-multilingual-MiniLM-L12-v2".

Au cours des différents tests, d'autres éléments ont été paramétrés :

Pour la vectorisation :

- ngram_range -> La taille des n-grams pris en compte
- min_df -> la fréquence minimale d'un terme pour qu'il soit pris en compte
- max_df -> fréquence maximale d'un terme pour qu'il soit pris en compte
- stop_words -> les stopwords français de la bibliothèque NLTK ont été inclus dès le départ. Nous avons ensuite ajouté une nouvelle liste de stopwords contenant principalement des noms d'articles, car les clusters se formaient en fonction du type d'article mentionné plutôt que des catégories souhaitées.

Pour l'algorithme de clustering :

- min_cluster_size -> la taille minimale qu'un cluster doit avoir
- min_samples -> densité minimale pour qu'un point soit considéré comme central

Pour BERTopic dans son ensemble :

- nr_topics -> réduction automatique du nombre de topics

Résultats et évaluation

Après un ajustement de ces paramètres, nous avons pu obtenir une trentaine de topics. Certains d'entre eux étaient très cohérents, tandis que d'autres étaient très mélangés. Les topics cohérents étaient relativement petits, alors que les autres étaient trop gros pour être exploitables. Une partie d'entre eux contenait simplement beaucoup d'avis très longs, sans prendre en compte leur contenu. Les avis courts, quant à eux, étaient bien répartis, tant qu'ils ne mentionnaient pas plusieurs thèmes à la fois.

Les résultats obtenus avec les métriques sont :

- Diversité : 0,78
- Cohérence (c_v) : 0,60

CTM (Contextualized Topic Models)

Principe du modèle

Le modèle **CTM (Contextualized Topic Models)** repose sur la combinaison de deux types de représentations complémentaires :

- une représentation **lexicale** de type Bag-of-Words (**BoW**),
- une représentation **contextualisée** basée sur des embeddings de type **BERT**.

Cette approche hybride permet de dépasser les limites des modèles probabilistes classiques tels que LDA, qui reposent uniquement sur la fréquence des mots, tout en conservant une **bonne interprétabilité des topics** grâce à la composante lexicale.

Prétraitement des données

Le prétraitement est volontairement **différencié selon le type de représentation**.

Pour la représentation lexicale (Bag-of-Words), un **filtrage des stopwords français**.

L'objectif est d'éviter que les topics soient dominés par des mots trop fréquents ou peu informatifs, afin de produire des thèmes plus lisibles et exploitables.

En revanche, ce filtrage n'est **pas appliqué à la représentation contextuelle**, car celle-ci doit conserver un maximum d'information sémantique. Le maintien du contexte est essentiel pour permettre au modèle de capturer les nuances et le sens global des avis.

Embeddings contextuels

Les embeddings sont générés à l'aide d'un modèle Transformer francophone entraîné pour produire des embeddings sémantiques de documents (dangvantuan/french-document-embedding).

Chaque avis est ainsi transformé en un vecteur dense de dimension fixe, représentant son contenu sémantique global.

Ce choix permet :

- une meilleure capture de la sémantique en langue française,
- une représentation plus robuste que **TF-IDF**, notamment pour des avis longs, nuancés ou exprimant des jugements complexes.

Préparation des données pour le modèle

La préparation des données est réalisée à l'aide de la classe **TopicModelDataPreparation**, qui permet de regrouper :

- la représentation lexicale (Bag-of-Words),
- la représentation contextuelle (embeddings SBERT),

au sein d'un **jeu de données d'entraînement unique** compatible avec le modèle CTM.

La tokenisation repose sur le modèle **CamemBERT** (camembert-base), adapté au français.

Entraînement du modèle

L'entraînement du modèle CTM nécessite notamment le paramétrage :

- du nombre de topics (***k***),
- du nombre d'époques d'apprentissage (***num_epochs***).

Plusieurs valeurs ont été testées, en particulier pour le nombre de topics :

- avec moins de 20 topics, les thèmes obtenus étaient trop généraux,
- avec plus de 40 topics, la fragmentation devenait excessive et certains topics apparaissent redondants.

Le choix final s'est donc porté sur **30 topics**, correspondant au meilleur compromis entre **granularité**, **cohérence** et **interprétabilité**.

Résultats et évaluation

Après extraction et analyse qualitative des topics, certains d'entre eux se sont révélés bien ciblés et cohérents, tandis que d'autres restaient plus mélangés et donc difficilement exploitables.

Afin de comparer les performances du modèle CTM avec les autres approches testées, deux métriques ont été utilisées :

- le **score de diversité des topics**,
- la **cohérence sémantique (c_v)**.

Pour le modèle CTM, les scores obtenus sont :

- **diversité : 0.80**,
- **cohérence (c_v) : 0.53**.

Ces résultats indiquent une amélioration notable par rapport aux modèles purement probabilistes, bien que certaines limites subsistent en termes de séparation claire entre certains topics.

TOP2VEC

Principe du modèle

Top2Vec est un modèle de topic modeling non supervisé, ce qui signifie qu'il n'est pas nécessaire de définir à l'avance le nombre de topics à extraire.

Le fonctionnement du modèle repose sur les étapes suivantes :

- Transformation de chaque document en un vecteur sémantique dense à l'aide d'un modèle d'embedding ;
- Projection des documents dans un espace sémantique commun ;
- Regroupement automatique des documents proches à l'aide des algorithmes **UMAP** et **HDBSCAN** ;
- Définition de chaque topic comme la moyenne vectorielle des documents qui lui sont associés.

Top2Vec est particulièrement adapté aux corpus de taille moyenne à grande, multilingues, et contenant des textes relativement riches sur le plan sémantique. La qualité des résultats dépend fortement du modèle d'embedding utilisé.

Prétraitement des données

Avant l'application du modèle de *topic modeling*, un prétraitement approfondi des données textuelles a été réalisé afin de réduire le bruit et d'améliorer la qualité des résultats. un prétraitement complet comme Mise en minuscules de tous les textes , Conservation des accents et des espaces ,Suppression des chiffres et de la ponctuation.

Suppression des stopwords: Une liste spécifique de mots peu informatifs, principalement liés à des catégories de produits, a été retirée du corpus.

Cette liste comprend notamment :

'montre', 'montres', 'boucles', 'oreilles', 'oreille', 'paire', 'paires',
'lunettes', 'bracelet', 'bracelets', 'collier', 'iphone', 'téléphone',
'pendentif', 'robot', 'robots', 'aspirateur', 'baskets', 'basket',
'chaussures', 'sandales', 'plantes', 'plante', 'arbres', 'arbre',
'bulbes', 'willemse', 'sommiers', 'jardin', 'bague', 'lampe', 'lampes',
'abat-jour', 'lampadaire', 'parfum', 'shampoing', 'shampooing',
'cheveux', 'masque', 'masques', 'crème', 'élastiques', 'manteau',
'bougie', 'cadre', 'écouteurs', 'vélo'

L'objectif de cette suppression est d'éviter que le modèle ne regroupe les commentaires uniquement selon des catégories de produits, plutôt que selon leur contenu sémantique.

Ces commentaires nettoyés ont ensuite été transformés en une liste de documents, qui constitue l'entrée du modèle Top2Vec.

Embeddings contextuels

Nous avons utilisé des embeddings contextuels basés sur le modèle 'distiluse-base-multilingual-cased'.

Ce type d'embedding permet de mieux capturer le sens sémantique des phrases, prendre en compte le contexte des mots dans les phrases . De plus, ce modèle est multilingue, ce qui le rend particulièrement adapté à l'analyse d'avis clients pouvant être rédigés dans différentes langues.

Entraînement du modèle

Le modèle Top2Vec est entraîné à partir de la liste des documents en utilisant les paramètres suivants :

- documents = docs : ensemble des commentaires nettoyés,
- embedding_model = 'distiluse-base-multilingual-cased' : modèle d'embedding multilingue,
- speed = 'learn' : mode d'apprentissage rapide,
- workers = 4 : utilisation de plusieurs cœurs du processeur.

Le modèle fournit le nombre de topics détectés ainsi que la taille (nombre de documents) associée à chaque topic.

Analyse et interprétation des résultats

Le modèle Top2Vec est mauvais en diversité (0.18) et en cohérence (0.39). Ces résultats s'expliquent par le fait que le modèle Top2Vec est plutôt adapté aux documents dans un corpus et moins à des textes courts comme dans notre cas.

LDA (Latent Dirichlet Allocation)

Principe du modèle

Le modèle LDA (Latent Dirichlet Allocation) est un algorithme de topic modeling non supervisé dont l'objectif est d'identifier automatiquement les thématiques principales présentes dans un corpus de textes.

Le principe fondamental de LDA repose sur les hypothèses suivantes :

- chaque document est composé de plusieurs topics ;
- chaque topic est une distribution de probabilités sur les mots ;
- Les mots observés dans un document sont générés à partir de ces distributions.

Prétraitement des données

Avant l'application du modèle LDA, un prétraitement complet comme:

- Conversion des textes en minuscules
- Suppression des URLs et des adresses email

- normalisation des espaces
- suppression des stopwords français
- filtrage des avis français à l'aide du modèle FastText de détection de langue

Embeddings Contextuels

Les textes ont été vectorisés à l'aide de TF-IDF(Term Frequency - Inverse Document Frequency) qui permet de représenter chaque document par un vecteur de poids de mots basé sur leur importance relative dans le corpus.

Les paramètres principaux utilisés sont:

- ngram range = (1,2) -> pour inclure les unigrams et bigrams
- min_df = 5 -> pour supprimer les termes trop rares
- max_df = 0.95 -> pour supprimer les termes trop fréquents et peu discriminants
- utilisation des stopwords français.

Entraînement du modèle

Le modèle LDA a été entraîné avec paramètres:

- Nombre de topics (n components)
- Paramètres de vectorisation TF-IDF(min_df et max_df pour filtrer les mots rares ou trop fréquents, suppression des stopwords français)
- Méthode d'apprentissage (online pour améliorer la stabilité et la scalabilité du modèle)

Le modèle a ensuite été entraîné avec 20 topics, cette étape permet au modèle d'apprendre la distribution des mots pour chaque topic et la distribution des topics pour chaque document.

Résultats et évaluation

Après un ajustement de ces paramètres, nous avons pu obtenir une 20 de topics.

Les résultats du modèle LDA montrent des performances contrastées:

- La **diversité des topics** est élevée (0.98), ce qui indique que les topics sont bien distincts les uns des autres au niveau lexical.
- En revanche, le **score de cohérence** (métrique c_v) est faible (0.35), ce qui signifie que les mots composant un même topic ne sont pas suffisamment liés sémantiquement.

Choix du modèle

Modèle	Diversité	Cohérence c_v
BERTopic	0.78	0.60
TOP2VEC	0.18	0.39
CTM	0.80	0.53
LDA	0.98	0.35

Nous avons d'abord retenu les modèles CTM et BERTopic, car leurs résultats étaient sensiblement meilleurs que ceux de LDA et TOP2VEC, notamment pour la cohérence, que nous avons privilégiée.

Nous avons tenté d'exploiter les 30 topics produits par CTM, mais trop peu de topics étaient réellement exploitables, car beaucoup étaient trop mélangés. Nous nous sommes donc tournés vers BERTopic, qui obtenait plus de topics cohérents.

Interprétation des résultats

Analyse des erreurs

Nous avons analysé les clusters créés par BERTopic, et en avons conclu qu'ils n'étaient pas très exploitables. Nous avons des petits clusters très cohérents, mais aussi deux très gros clusters hétérogènes, d'autres clusters plus petits mais hétérogènes également, et nous avons remarqué que les avis longs étaient regroupés ensemble, indépendamment de ce dont ils parlaient. Les deux gros clusters étaient le premier, considéré comme un vrai cluster par le modèle, ainsi que le cluster -1 : celui qui regroupe tous les outliers. Le modèle n'a donc pas réussi à regrouper ces avis en clusters cohérents et a préféré les garder à part.

Finalement, le nombre d'avis dans les clusters cohérents n'était pas suffisamment élevé pour être exploitable.

Amélioration du modèle

Cette analyse nous a permis d'apporter plusieurs modifications à notre pipeline. Tout d'abord, afin de diviser les clusters hétérogènes, nous avons augmenté le nombre de clusters.

Les deux très gros clusters étant beaucoup trop gros pour être exploitables (autour de 6000 avis pour chaque topics, alors que le dataset en contient 15000 au total), nous avons utilisé la technique des K-means pour les diviser. Nous avons donc ajouté une nouvelle étape dans la pipeline, où les topics contenant plus de 2000 commentaires étaient divisés avec K-means. Cet algorithme, nécessitant un nombre de clusters, nous avons indiqué vouloir environ 200 documents par nouveau cluster. Cela nous a permis d'atteindre un total de 100 clusters.

De plus, comme nous avons remarqué que quelques clusters regroupaient les avis très longs entre eux, nous avons ajouté une étape de pré-processing, en segmentant les avis par phrases à l'aide de Spacy. Cette étape nous a fait passer de 15073 à 37539 données.

Suite à ces changements, nous avons obtenu 200 clusters, qui semblaient beaucoup plus cohérents. Une partie des clusters était toujours hétérogène, mais beaucoup plus de commentaires étaient exploitables après ces modifications.

Méthode d'interprétation

Pour interpréter les résultats, nous avons analysé manuellement les clusters : nous avons regardé les listes de top words par clusters générées par le modèle, mais nous avons pu noter que les top words ne sont pas toujours très représentatifs d'un cluster : un mot ou expression peut apparaître plusieurs fois dans quelques avis, et les autres avis peuvent ne pas du tout parler de la même chose.

Nous avons également regardé les avis regroupés dans chaque clusters afin d'évaluer si ceux-ci étaient cohérents ou non. Cette méthode était bien plus efficace, bien que très chronophage. Elle nous aura permis d'avoir une idée beaucoup plus précise des erreurs et des problèmes à résoudre.

Annotation manuelle des clusters

Une fois les clusters obtenus, l'étape suivante était de les annoter avec les trois catégories : qualité produit, service livraison, service client. Nous nous sommes donc répartis les topics à hauteur de 50 par personne. Pour annoter les clusters, nous nous sommes aidés de la représentation des topics avec les top words, ainsi que des phrases (avis segmentés en phrases) qui forment chaque topic. Selon les topics, plusieurs cas se sont dessinés :

- Le topic est homogène et ne mentionne qu'une seule catégorie. C'est un topic idéal, l'annotation est simple, avec une seule catégorie.
Ex : qualité produit
- Le topic est hétérogène et mentionne deux ou trois catégories. Dans ce cas, deux possibilités : soit les avis parlent de plusieurs thèmes (ex : "Très bon produit, livraison rapide"), soit des avis parlant de thèmes différents sont dans le même cluster (ex : "Livraison rapide" et "Le service client n'a pas répondu à ma demande"). Dans les deux cas, on annote en indiquant les différentes catégories présentes dans le topic.
Ex : qualité produit / service livraison

- Le topic ne mentionne aucune catégorie (ex : "Je suis très satisfait de ma commande."). Ces avis ne nous intéressent pas, on annote avec "Neutre".

Nous avons ainsi obtenu des labels pour chacun de nos clusters, et de ce fait, pour chacune des phrases segmentées. Nous n'avons gardé que les clusters annotés avec une seule catégorie, les autres n'étant pas suffisamment exploitables pour la suite du projet. Ensuite, en rassemblant les phrases appartenant à un même commentaire, nous avons ainsi obtenu des commentaires annotés en multi-label avec nos trois catégories, sous la forme de one-hot encoding.

Nous obtenons donc un dataset de 6535 commentaires annotés, sur les 15073 à annoter. Afin d'annoter les 8538 avis restants, nous avons décidé d'entraîner un modèle de classification à partir des avis déjà annotés.

II. Annotation weakly-supervisée* par règles linguistiques

*On parle de méthode weakly-supervisée car les labels sont générés automatiquement à partir de règles définies manuellement, et non par une annotation humaine. Les signaux utilisés pour annoter les données sont donc imparfaits, mais suffisants pour constituer un jeu de données d'entraînement exploitable pour une tâche de classification supervisée.

Contexte et motivation

Les différentes approches de topic modeling testées au cours du projet (LDA, CTM, BERTopic, Top2Vec) ont permis d'explorer la structure thématique du corpus et d'annoter une partie des avis. Toutefois, plusieurs limites ont été identifiées :

- une proportion importante d'avis restait non labellisée,
- certains clusters obtenus étaient hétérogènes ou difficiles à interpréter,
- l'annotation manuelle des clusters s'est révélée coûteuse en temps.

Dans un contexte où l'objectif principal du projet est la **création de données labellisées fiables** afin d'entraîner des modèles de classification multi-label, nous avons exploré une approche complémentaire d'annotation, basée sur des règles linguistiques explicites.

Principe de l'approche

Cette méthode repose sur une **annotation weakly-supervisée par règles**, utilisant des **expressions régulières** (REGEX) afin de détecter la présence de mots ou suite de mots dans chacune des catégories cibles :

- qualité produit,
- service livraison,
- service client.

Contrairement au topic modeling, cette approche ne cherche pas à découvrir automatiquement des thèmes latents, mais à **identifier explicitement des signaux sémantiques forts** dans les commentaires, à partir de connaissances métier et d'une analyse qualitative préalable du corpus.

Chaque commentaire peut se voir attribuer **un ou plusieurs labels**, ce qui correspond naturellement au cadre de la classification multi-label.

Construction des règles linguistiques

Pour chaque catégorie, une liste de mots-clés et d'expressions a été définie manuellement à partir :

- de l'analyse des clusters issus du topic modeling,
- de l'inspection directe des avis,
- des formulations récurrentes observées dans le corpus.

Ces listes ont ensuite été formalisées sous forme d'expressions régulières prenant en compte :

- les variantes morphologiques (accords, conjugaisons),
- les variantes orthographiques,
- certaines ambiguïtés contextuelles.

Par exemple, pour la catégorie qualité produit, des règles spécifiques ont été mises en place afin d'éviter la confusion avec des expressions relevant du service client ou de la livraison.

Processus d'annotation

Le processus d'annotation suit les étapes suivantes :

1. chargement du corpus de commentaires prétraités,
2. application des expressions régulières à chaque commentaire,
3. attribution d'un label binaire par catégorie (présence / absence),
4. conservation des commentaires contenant au moins un label.

Cette méthode permet une annotation entièrement déterministe, rapide à exécuter et totalement interprétable.

Résultats obtenus

L'annotation par règles a permis de labelliser une proportion significative du corpus, avec :

- une meilleure couverture de certains thèmes sous-représentés par le topic modeling,
- une répartition des classes plus équilibrée,
- un nombre important de commentaires annotés en multi-label.

Ces résultats en font une source de labels exploitable pour l'entraînement des modèles de classification, en complément des données issues du topic modeling.

Voici la répartition des classes obtenues avec cette méthode :

	nb_avis_annotés
qualité produit	5340
service livraison	6971

service client	4569
----------------	------

Au total, avec cette méthode, on obtient 11449 avis qui possèdent au moins 1 label.

Avantages et limites

Avantages :

- méthode simple et rapide à mettre en œuvre,
- absence de phase d'entraînement,
- interprétabilité totale des décisions,
- contrôle précis sur les catégories détectées.

Limites :

- dépendance au vocabulaire explicitement présent dans les règles,
- difficulté à détecter des formulations implicites ou ironiques,
- nécessité d'un ajustement manuel des règles pour améliorer la couverture.

Dans le cadre de ce projet, ces limites sont acceptables, car l'objectif n'est pas de produire une annotation exhaustive, mais de **constituer un jeu de données d'entraînement fiable**, avec un bon compromis entre précision et couverture.

Positionnement dans le pipeline global

Cette approche d'annotation s'inscrit comme une **stratégie complémentaire** (ou alternative, au choix) au topic modeling.

Le topic modeling a principalement servi à explorer le corpus et à identifier des thématiques, tandis que l'annotation par règles a permis de produire des labels plus directement exploitables pour la phase de classification supervisée.

Cependant, ayant testé cette solution assez tardivement dans le projet, nous avons fait l'étape suivante de classification supervisée avec les données annotées via le topic modeling suivi de l'annotation manuelle. Les données obtenues avec cette technique ont été utilisées uniquement sur le modèle de régression logistique.

III. Classification multi-label

Classification du problème

La deuxième partie qui s'est imposée pour ce projet consiste à annoter les 8538 avis non labellisés en utilisant les 6535 avis labellisés grâce au topic modeling. Il s'agit donc d'une tâche de classification multi-label, avec trois classes : qualité produit, service livraison, service client.

Nous avons utilisé plusieurs métriques : l'accuracy, le rappel, la précision, et le f-score. Nous avons privilégié le f-score, car celui-ci balance le rappel et la précision, tandis que l'accuracy ne prend pas en compte les déséquilibres au sein d'une classe. Nous avons également affiché des matrices de confusion pour chacune des trois classes.

Deux tests ont été effectués pour chaque modèle testé. Nous les avons d'abord évalués sur un set de test correspondant à 20% des données annotées via le topic modeling. Nous avons utilisé l'argument "seed" afin que tous les modèles soient testés sur le même set de test. Ce set est donc très similaire à celui de train, et devrait donc obtenir de bons résultats.

Nous avons également testé nos modèles sur un set de 100 avis annotés manuellement par les membres du groupe. Ceux-ci ont été sélectionnés parmi ceux qui n'avaient pas pu être annotés via le topic modeling ni la technique des regex : ils font donc partie des avis difficiles à classer, et ne ressembleront peut-être pas aux données d'entraînement. Il est probable que ces avis soient plus longs ou plus ambigus dans le cadre de la détection des labels.

Nous avons essayé cinq algorithmes différents : XGBoost, CamemBERT fine-tuné pour la classification, KNN, SVM, et la régression logistique. Pour chaque modèle, nous allons présenter son fonctionnement, le prétraitement nécessaire, l'ajustement qui a été effectué, ainsi que les résultats obtenus.

XGBoost

Principe du modèle

Le modèle XGBoost (eXtreme Gradient Boosting) est basé sur le principe du gradient boosting d'arbres de décision. Celui-ci consiste à construire une série d'arbres de décision, chacun corrigeant les erreurs du précédent, en utilisant les gradients de la fonction de perte ainsi qu'une régularisation. En classification, une prédiction du modèle correspond à la probabilité que la classe soit présente. Il est donc possible de modifier le seuil de décision, qui est fixé à 0,5 par défaut.

Il s'agit dans notre cas de classification multi-label. XGBoost ne gère pas nativement le multilabel, cependant il est possible de l'appliquer, à l'aide de la méthode One-vs-Rest. Celle-ci consiste à entraîner un modèle XGBoost binaire par label, où chaque modèle prédira la probabilité de présence du label correspondant.

Prétraitement et embeddings

Nous avons effectué le prétraitement cité dans le rapport de preprocessing pour les embeddings de phrases. Quatre embeddings différents ont été testés : "paraphrase-multilingual-MiniLM-L12-v2" , "all-mpnet-base-v2", "all-MiniLM-L6-v2", et "camembert-base". Les trois premiers sont des embeddings multilingues, tandis que le dernier est français.

Ajustement du modèle

Le modèle a été testé à l'aide de la cross validation (5 folds du dataset de train), afin de sélectionner deux paramètres : les embeddings, et la profondeur maximale "max_depth", allant de 3 à 5. L'embedding retenu est "camembert-base", tandis que la profondeur maximale est de 5. Au vu de ce résultat, nous aurions peut-être pu augmenter cette variable, mais nous ne sommes pas allés plus loin pour des raisons de puissance de calcul, cette profondeur étant déjà relativement longue à entraîner.

Résultats et évaluation

Après un entraînement final avec les paramètres retenus, nous avons testé le modèle sur un dataset de test correspondant à 20% du dataset annoté à l'aide du topic modeling. Voici les résultats obtenus :

Label	Précision	Rappel	F-score
Qualité produit	0,77	0,87	0,82
Service livraison	0,80	0,67	0,73
Service client	0,70	0,28	0,40
Micro avg	0,78	0,71	0,74
Macro avg	0,76	0,61	0,65

On peut tout d'abord observer que le rappel pour le label "service client" est très mauvais, ce qui fait chuter le f-score de cette classe. Cela peut s'expliquer par le fait que la répartition entre les classes est déséquilibrée au niveau du service client, qui est beaucoup moins représenté que les deux autres classes. Les résultats pour le label "qualité produit" sont plutôt bons, et ceux de "service livraison" aussi bien qu'un peu moins car le rappel est plus bas.

Au niveau de la moyenne entre les classes, la répartition entre les classes étant déséquilibrée, nous nous intéressons surtout à la moyenne macro, qui est plus basse que la micro car le label "service client" prend autant d'importance malgré sa sous-représentation. Le f-score moyen final s'élève donc à 0,65.

La matrice de confusion ci-dessous reflète également ces résultats.

Matrice de confusion – qualité produit

	Prédit 0	Prédit 1
Vrai 0	405	185
Vrai 1	95	622

Matrice de confusion – service livraison

	Prédit 0	Prédit 1
Vrai 0	670	90
Vrai 1	180	367

Matrice de confusion – service client

	Prédit 0	Prédit 1
Vrai 0	1073	25
Vrai 1	150	59

Nous avons également testé le modèle sur un dataset de 100 avis annotés manuellement par les membres du groupe, afin de comparer les résultats, et de voir si les données annotées via le topic modeling sont fiables. Voici les résultats obtenus :

Label	Précision	Rappel	F-score
Qualité produit	0,55	0,78	0,65
Service livraison	0,57	0,62	0,59
Service client	0,33	0,04	0,07
Micro avg	0,55	0,52	0,53
Macro avg	0,48	0,48	0,43

Matrice de confusion – qualité produit

	Prédit 0	Prédit 1
Vrai 0	33	26
Vrai 1	9	32

Matrice de confusion – service livraison

	Prédit 0	Prédit 1
Vrai 0	69	10
Vrai 1	8	13

Matrice de confusion – service client

	Prédit 0	Prédit 1
Vrai 0	71	2
Vrai 1	26	1

On peut observer que les résultats sont très mauvais pour le label "service client", et juste bons pour les deux autres labels. Le f-score moyen se situe à 0,43.

CamemBERT fine-tuné

Principe du modèle

CamemBERT est un modèle de type Transformer basé sur RoBERTa, et pré-entraîné sur de grands corpus français. Pour l'entraîner sur une tâche de classification multilabel, il faut le fine-tuner. Cela signifie qu'on ajoute un élément indiquant que la tâche est la classification, avant de l'entraîner sur cette tâche.

Contrairement aux autres modèles testés, celui-ci permet une compréhension sémantique du texte et ne nécessite aucun prétraitement ni embedding préalable. Il est également capable d'apprendre les corrélations entre les labels, en les apprenant simultanément.

Ce modèle complexe comprend plusieurs étapes : il commence par une tokenisation du texte à partir du tokenizer interne à CamemBERT, avant d'intégrer ce résultat à CamemBERT, qui crée son propre embedding. Celui-ci passe ensuite dans une couche de classification, et sort en output un vecteur dont la taille correspond au nombre de labels, où chaque valeur est une probabilité.

Prétraitement et embeddings

Nous avons gardé le prétraitement cité dans le rapport de preprocessing pour les embeddings de phrases, mais en utilisant directement le texte au lieu des embeddings.

Ajustement du modèle

Après une sélection des hyperparamètres à l'aide de la cross-validation, nous avons entraîné le modèle en utilisant un dataset de validation, selon cette répartition : 20% pour le test, 15% pour la validation, et 65% pour le train.

Une fois le modèle entraîné, nous avons cherché à améliorer les résultats en modifiant le seuil de prédiction, qui est fixé à 0,5 par défaut. Nous avons optimisé les seuils indépendamment pour chaque label à l'aide de la méthode GridSearch, sur le jeu de validation.

Résultats et évaluation

Le modèle a ensuite été testé avec les seuils obtenus sur le dataset de test correspondant à 20% du dataset annoté. Voici les résultats obtenus :

Label	Précision	Rappel	F-score
Qualité produit	0,81	0,84	0,83
Service livraison	0,73	0,81	0,77
Service client	0,47	0,58	0,52
Micro avg	0,73	0,80	0,76
Macro avg	0,67	0,74	0,70

Matrice de confusion – qualité produit

	Prédit 0	Prédit 1
Vrai 0	413	145
Vrai 1	121	628

Matrice de confusion – service livraison

	Prédit 0	Prédit 1
Vrai 0	596	162
Vrai 1	105	444

Matrice de confusion – service client

	Prédit 0	Prédit 1
Vrai 0	1003	119
Vrai 1	78	107

Les résultats sont plutôt très bons, avec toujours une faiblesse pour la classe "service client", qui s'explique par le déséquilibre des classes dans les données d'entraînement. Cela peut aussi s'expliquer par le fait que cette classe est plus difficile à labelliser manuellement, car elle est exprimée de manière plus ambiguë. On obtient au final un f-score de 0,70.

Nous avons ensuite testé ce modèle sur le set de 100 avis annotés manuellement :

Label	Précision	Rappel	F-score
Qualité produit	0,58	0,93	0,71
Service livraison	0,37	0,76	0,50
Service client	0,40	0,15	0,22
Micro avg	0,49	0,65	0,56
Macro avg	0,45	0,61	0,48

Matrice de confusion – qualité produit

	Prédit 0	Prédit 1
Vrai 0	31	28
Vrai 1	3	38

Matrice de confusion – service livraison

	Prédit 0	Prédit 1
Vrai 0	52	27
Vrai 1	5	16

Matrice de confusion – service client

	Prédit 0	Prédit 1
Vrai 0	67	6
Vrai 1	23	4

Sur les 100 avis, on trouve toujours de mauvais résultats pour la classe "service client", mais on peut aussi remarquer que la précision est beaucoup plus basse que le rappel pour les deux autres classes. Beaucoup d'avis sont donc des faux positifs, ce qui signifie que le modèle a tendance à voir ces classes dans des avis où elles ne sont pas mentionnées. Le f-score final s'élève à 0,48.

KNN

Principe du modèle

Modèle KNN c'est l'algorithme des k plus proches voisins (KNN) est un **classificateur d'apprentissage non paramétrique et supervisé** qui s'appuie sur la notion de proximité pour réaliser des classifications ou des prédictions sur le regroupement d'un point de données. Il s'agit d'une des méthodes de classification et de régression les plus simples et les plus utilisées actuellement dans le machine learning.

Prétraitement et embeddings

Nous avons gardé le prétraitement cité dans le rapport de preprocessing pour les embeddings de phrases, mais en utilisant directement le texte au lieu des embeddings

Ajustement du modèle

Dans le modèle KNN, il n'y a pas `class_weight = 0`. Nous avons préféré ajouter un argument `weights = distance` qui permet de limiter le déséquilibre des classes.

Évaluation et validation

Le modèle a ensuite été testé avec les seuils obtenus sur le dataset de test correspondant à 20% du dataset annoté. Voici les résultats obtenus :

ML KNN	Precision	Recall	F1-score
Qualité produit	0.74	0.78	0.76
Service livraison	0.72	0.63	0.67
Service client	0.51	0.16	0.24
Micro avg	0.69	0.69	0.69
Macro avg	0.64	0.54	0.60

```

Matrice de confusion pour qualité produit
      Prédit 0  Prédit 1
Vrai 0      359      231
Vrai 1      126      591

Matrice de confusion pour service livraison
      Prédit 0  Prédit 1
Vrai 0      588      172
Vrai 1      168      379

Matrice de confusion pour service client
      Prédit 0  Prédit 1
Vrai 0     1049       49
Vrai 1      157       52

```

Nous avons ensuite testé ce modèle sur le set de 100 avis annotés manuellement :

Résultat de ces métriques :

ML KNN	Precision	Recall	F1-score
Qualité produit	0.61	0.61	0.61
Service livraison	0.12	0.05	0.07
Service client	0.45	0.52	0.48
Micro avg	0.50	0.45	0.47
Macro avg	0.40	0.39	0.31

```

Matrice de confusion pour qualité produit
      Prédit 0  Prédit 1
Vrai 0       43       16
Vrai 1       16       25

Matrice de confusion pour service livraison
      Prédit 0  Prédit 1
Vrai 0       72        7
Vrai 1       20        1

Matrice de confusion pour service client
      Prédit 0  Prédit 1
Vrai 0       56       17
Vrai 1       13       14

```

SVM

Principe du modèle

SVM(Support Vector Machine linéaire), c'est algorithm est bien adapté aux données textuelles vectorisées en TF-IDF , sa capacité à gérer des espaces de grande dimension et son efficacité dans des contextes comportant un grand nombre de classes et un fort déséquilibre des données.

Le modèle Linear SVC ne permet que la classification binaire mono-label , dans notre cas, un avis peut appartenir à plusieurs catégories en même temps (ex : qualité produit + service livraison).

Pour gérer cela, nous avons utilisé l'approche One-vs-Rest (OvR) via OneVsRest Classifier. Un classifieur SVM est entraîné indépendamment pour chaque label. Chaque modèle prédit la présence (1) ou l'absence (0) de la catégorie. Les prédictions sont ensuite combinées pour produire une sortie multilabel. Les catégories étudiées sont: qualité produit, service livraison, service client.

Prétraitement et embeddings

Avant l'entraînement du modèle, un prétraitement textuel a été appliqué aux avis afin d'améliorer la qualité des données .

Les textes nettoyés ont ensuite été représentés numériquement à l'aide de la vectorisation TF-IDF (Term Frequency – Inverse Document Frequency). Nous avons effectué le prétraitement décrit dans le rapport de preprocessing pour les embedding TF-IDF.

Ajustement du modèle

Le modèle utilisé est un LinearSVC, intégré dans un OneVest Classifier.

deux métriques ont été utilisées :

- Class_weight = 'balanced' : Permet de gérer le déséquilibre entre les classes en pénalisant davantage les erreurs sur les classes minoritaires.
- max_iter = 5000 : Augmente le nombre maximal d'itérations afin d'assurer la convergence du modèle.

Évaluation et validation

Une validation croisée à 5 plis a été réalisée afin d'évaluer la robustesse et la stabilité du modèle avant l'entraînement final et l'évaluation sur le jeu de test.

Nous avons testé le modèle sur un dataset de test correspondant à 20% du dataset annoté à l'aide du topic modeling.

Un **classification report** a permis d'analyser les performances par catégorie (précision, rappel, F1-score).

- Résultat de ces métriques :

SVM	Precision	Recall	F1-score
Qualité produit	0.66	0.73	0.69
Service livraison	0.68	0.62	0.65
Service client	0.42	0.59	0.49
Micro avg	0.63	0.66	0.64
Macro avg	0.59	0.64	0.61

Confusion matrix for qualité produit:

	Prédit 0	Prédit 1
Vrai 0	235	85
Vrai 1	61	165

Confusion matrix for service livraison:

	Prédit 0	Prédit 1
Vrai 0	181	81
Vrai 1	109	175

Confusion matrix for service client:

	Prédit 0	Prédit 1
Vrai 0	409	62
Vrai 1	31	44

Le modèle donne de bonnes performances pour la qualité du produit et le service de livraison, avec des F1-scores , 0.69 et 0.65.

En revanche, les performances sont plus faibles pour le service client(0.49), ce qui montre une difficulté du modèle à identifier correctement cette catégorie. Les matrices de confusion montrent que le modèle prédit globalement mieux les avis négatifs que les avis positifs.

On a également testé le modèle sur un ensemble de données de 100 avis annotés manuellement afin de comparer les résultats et de vérifier si les données annotées à l'aide de la Topic modelling sont fiables.

Résultat :

SVM	Precision	Recall	F1-score
Qualité produit	0.36	0.20	0.25
Service livraison	0.25	0.62	0.36
Service client	0.15	0.11	0.13
Micro avg	0.26	0.27	0.26
Macro avg	0.26	0.31	0.25

Matrice de confusion – qualité produit

	Prédit 0	Prédit 1
Vrai 0	45	14
Vrai 1	33	8

Matrice de confusion – service livraison

	Prédit 0	Prédit 1
Vrai 0	41	38
Vrai 1	8	13

Matrice de confusion – service client

	Prédit 0	Prédit 1
Vrai 0	56	17
Vrai 1	24	3

On trouve bien les résultats F1 score pour les label “service livraison”

Régression Logistique

Principe du modèle

La **régression logistique** est un modèle de classification linéaire supervisé qui estime, pour chaque classe, la probabilité d'appartenance d'un échantillon à cette classe à l'aide d'une fonction sigmoïde.

Bien que simple, ce modèle est largement utilisé en traitement automatique du langage naturel en raison de sa **robustesse**, de sa **bonne capacité de généralisation** et de sa **facilité d'interprétation**.

Dans le cadre d'une classification **multilabel**, où un même avis peut appartenir simultanément à plusieurs catégories (qualité produit, service livraison, service client), l'approche One-vs-Rest (OvR) avec OneVsRestClassifier a été retenue.

Un classifieur binaire indépendant est entraîné pour chaque label, permettant de prédire séparément la présence ou l'absence de chaque catégorie.

Prétraitement et embeddings

Les avis textuels ont été prétraités selon le pipeline décrit dans la section de preprocessing (nettoyage, normalisation linguistique).

Chaque avis a ensuite été représenté sous la forme d'un embedding dense de document, généré à l'aide du modèle Transformer francophone dangvantuan/french-document-embedding.

Ce modèle produit une représentation vectorielle capturant le sens global du document, ce qui permet à la régression logistique de travailler sur un espace sémantique dense plutôt que sur une représentation sparse de type TF-IDF.

Les embeddings ont été normalisés afin de faciliter l'apprentissage du modèle linéaire.

Ajustement du modèle

Le classifieur utilisé est une **régression logistique avec pénalisation L2**, intégrée dans un schéma One-vs-Rest.

Les principaux hyperparamètres sont les suivants :

- `penalty = "l2"` : régularisation pour limiter le surapprentissage,
- `solver = "lbfgs"` : solveur adapté aux problèmes multiclassés,
- `max_iter = 1000` : augmentation du nombre d'itérations pour assurer la convergence,
- `class_weight = "balanced"` : prise en compte du déséquilibre entre les classes.

Le jeu de données annoté a été séparé en **80 % pour l'entraînement et 20 % pour le test**.

Le modèle entraîné est ensuite sauvegardé afin d'être réutilisé sans réentraînement.

Résultats et évaluations

Les performances obtenues sur le jeu de test sont les suivantes :Le modèle obtient de **bons résultats globaux**, en particulier pour les classes *qualité produit* et *service livraison*.

La classe *service client* présente un rappel élevé mais une précision plus faible, indiquant une tendance du modèle à **sur-prédire cette catégorie**, probablement en raison du déséquilibre des classes et de la proximité sémantique avec les autres labels.

Les performances obtenues sur le jeu de test sont les suivantes :

Regresssion Logistique	Precision	Recall	F1-score
Qualité produit	0.79	0.79	0.79
Service livraison	0.73	0.74	0.74
Service client	0.41	0.78	0.53
Micro avg	0.68	0.77	0.72
Macro avg	0.64	0.77	0.69

Le modèle obtient de **bons résultats globaux**, en particulier pour les classes *qualité produit* et *service livraison*.

La classe service client présente un rappel élevé mais une précision plus faible, indiquant une tendance du modèle à **sur-prédire cette catégorie**, certainement en raison du déséquilibre des classes et de la proximité sémantique avec les autres labels.

Matrices de confusion :

Service livraison	Prédit 0	Prédit 1
Vrai 0	611	149
Vrai 1	141	406

Service client	Prédit 0	Prédit 1
Vrai 0	863	235
Vrai 1	47	162

Qualité produit	Prédit 0	Prédit 1
Vrai 0	441	149
Vrai 1	149	568

Les performances obtenues sur les 100 avis annotés manuellement sont les suivantes :

Regresssion Logistique	Precision	Recall	F1-score
Qualité produit	0.61	0.85	0.71

Service livraison	0.53	0.76	0.63
Service client	0.44	0.30	0.36
Micro avg	0.56	0.66	0.61
Macro avg	0.53	0.64	0.57

Sur ce jeu de données, les performances restent **globalement cohérentes**, le rappel reste assez élevé sauf pour le service client.

La baisse du rappel pour la classe *service client* suggère une **difficulté accrue à généraliser cette catégorie**, confirmant son caractère plus ambigu et subjectif.

Matrices de confusion :

Qualité produit	Prédit 0	Prédit 1
Vrai 0	37	22
Vrai 1	6	35

Service client	Prédit 0	Prédit 1
Vrai 0	63	10
Vrai 1	19	8

Service livraison	Prédit 0	Prédit 1
Vrai 0	65	14
Vrai 1	5	16

Interprétation des résultats

Dataset de test 20% :

Modèle	Rappel	Précision	F-score
KNN	0.64	0.54	0.60
SVM	0.64	0.59	0.61
Régression logistique	0.77	0.64	0.69
XGBoost	0.61	0.76	0.65
CamemBERT fine-tuné	0.74	0.67	0.70

Dataset annoté manuellement :

Modèle	Rappel	Précision	F-score
KNN	0.39	0.40	0.31
SVM	0.27	0.26	0.25
Régression logistique	0.64	0.53	0.57
XGBoost	0,48	0,48	0,43
CamemBERT fine-tuné	0,61	0,45	0,48

De manière générale, pour tous les modèles, on peut constater des performances moins bonnes pour la classe "service client". Cela s'explique principalement par un déséquilibre entre les classes, qui a pu biaiser l'entraînement des modèles, qui ne disposaient pas de suffisamment d'exemples pour identifier cette classe. Cependant, nous avons aussi remarqué, lors de l'étude manuelle des avis, que les avis traitant du service client étaient généralement très longs, ce qui rend l'étape d'encodage du texte plus difficile. De l'information a pu être perdue lors de cette étape. Nous avons aussi noté que cette classe semble plus ambiguë : même par

un humain, il est plus difficile d'identifier des termes ou expressions traitant systématiquement du service client, plutôt que de la qualité produit ou du service livraison.

Nous avons également pu remarquer que les résultats étaient nettement moins bons sur le dataset annoté manuellement que sur le dataset de test annoté via le topic modeling. Cela peut paraître étrange, mais nous y avons trouvé plusieurs explications. Tout d'abord, les avis utilisés pour l'entraînement et pour l'évaluation sont constitués différemment : le dataset annoté manuellement est constitué d'avis qui n'ont pas pu être annotés avec les techniques utilisées (topic modeling et regex). Ces avis sont donc intrinsèquement difficiles à classer. Il est également possible qu'ils ne ressemblent pas aux données d'entraînement des modèles (qui ont été annotées via le topic modeling) : ils peuvent être plus longs, plus ambigus, ou utiliser un vocabulaire plus spécifique, qui n'a pas été associé à un label par les modèles.

De plus, l'annotation est également très différente entre les données d'entraînement de ce dataset de test. L'annotation manuelle ayant été faite par un humain, il est possible qu'elle n'ait rien à voir avec celle obtenue via le topic modeling. Les règles d'annotation que nous établissons inconsciemment sont celles que nous souhaitons que le modèle trouve, mais ce ne sont pas celles qu'il établira. Nous utilisons des biais que le modèle est incapable de trouver. Aussi, nous avons nous-mêmes eu des difficultés à annoter certains avis ambigus, nous ne pouvons donc pas attendre du modèle de topic modeling qu'il réussisse parfaitement la tâche.

Ces différences au niveau des données et de l'annotation engendrent deux datasets très différents. Le dataset de test ressemblant aux données d'entraînement sur ces deux points, il est normal qu'il obtienne de meilleurs résultats que le dataset annoté manuellement.

Conclusion

Dans ce projet, nous avons cherché à concevoir un modèle de classification automatique pour extraire les informations utiles des avis clients TrustPilot, et les attribuer à trois classes distinctes : qualité produit, service livraison, service client.

Pour cela nous avons effectué un preprocessing sur la variable commentaire, en utilisant différents embeddings de texte : TF-IDF, Word2Vec, Sentence-BERT multilingue, et CamemBERT.

Ensuite, pour regrouper nos commentaires par thématiques, nous avons testé quatre différents modèles de topic modeling : LDA, Top2Vec, CTM, et BERTopic. BERTopic obtenant le plus de topics cohérents, il a été sélectionné pour la suite.

Ayant obtenu deux gros clusters peu exploitables, nous avons utilisé la technique des K-means pour les rediviser. Nous avons également utilisé Spacy pour rediviser les avis par phrases.

Ces changements nous ont permis d'obtenir 200 clusters, que nous avons annotés manuellement dans les trois catégories évoquées ci-dessus : qualité produit, service livraison, service client.

Les clusters annotés à une seule catégorie ont été gardés. En rassemblant les phrases appartenant à un même avis, nous avons obtenu un dataset de commentaires annotés en multilabel.

Nous avons par la suite utilisé cinq algorithmes adaptés à la classification multi-label : XGBoost, CamemBERT fine-tuné pour la classification, KNN, SVM, et la régression logistique.

Le dataset d'entrée pour ces algorithmes correspond à une partie de celui qui a été obtenu via le topic modeling (process développé au-dessus).

Nous avons testé ces cinq algorithmes avec deux datasets : la partie non utilisée pour l'entraînement de celui qui a été obtenu via le topic modeling, et un jeu de données de 100 avis annotés manuellement.

Les résultats de prédictions sont assez similaires pour tous les algorithmes même si les modèles Camembert fine-tuné, XGBOOST et Régression logistique semblent avoir des meilleurs résultats de prédictions. Pour tous les modèles de classification la classe « Service client » est difficilement prévisible ayant un caractère ambigu.

Nous avons rencontré plusieurs difficultés au cours de ce projet. Le premier obstacle auquel nous avons été confrontés a été le besoin d'annoter un dataset non labellisé. Notre objectif était au départ d'entraîner un modèle de classification, mais au vu des nombreuses difficultés, nous nous sommes concentrés sur la labellisation des données. Le topic modeling devait nous permettre de trouver les principaux thèmes, qui seraient nommés à l'aide de techniques statistiques. Malgré les nombreux modèles utilisés, aucun n'a donné de topics parfaits. Les clusters étaient trop hétérogènes et difficilement interprétables, ce qui a nécessité plusieurs ajustements non prévus, qui ont pris beaucoup de temps.

Nous avons dû labelliser les topics obtenus à la main, car les techniques statistiques ne donnaient pas de bons résultats. Cette étape, bien qu'effectuée par des humains, et donc plus fiable que les modèles de topic modeling, a été relativement difficile, à cause de l'hétérogénéité des clusters, mais aussi parce que nous

avons chacun nos propres règles implicites. L'annotation des 100 avis annotés qui ont permis d'évaluer les modèles de classification a été tout aussi difficile, notamment parce que nous n'avions pas établi de règles d'annotation au préalable. Avec le recul, nous pensons qu'il aurait été intéressant de prendre le temps d'annoter quelques centaines d'avis manuellement dès le début du projet, afin de mieux comprendre nos données. Nous aurions par exemple pu remarquer que certains avis très longs pourraient poser problème, et donc décidé dès le départ de les segmenter en phrases. Cette étape nous aurait également aidés à déterminer les thèmes principaux présents dans nos données, ainsi qu'à établir un guide d'annotation, que l'on aurait pu utiliser pour l'alternative d'annotation weakly-supervisée par regex. Enfin, nous aurions disposé d'un dataset de test dès le départ, que nous aurions pu utiliser pour évaluer les modèles de topic modeling.

Les classes choisies ont également causé des difficultés, car parfois il était difficile de savoir laquelle était vraiment citée. De plus, la classe "service client" peut s'avérer très ambiguë. Les modèles ont eu beaucoup de mal à identifier cette classe, car elle correspond généralement à des avis très longs, que nous avons nous-mêmes eu du mal à classer. Il aurait peut-être fallu définir nos trois classes autrement, afin de faciliter l'identification des classes par les modèles, en utilisant des classes que nous savons facilement identifier.

Nous avons également rencontré quelques difficultés à l'étape du preprocessing. Ne sachant pas encore exactement quels modèles nous allions utiliser par la suite, nous avons préparé beaucoup d'embeddings différents, que nous n'avons pas tous utilisés lors de l'étape de modélisation.

Nous avons aussi fait face à des difficultés techniques : le manque de GPU a rendu la création de certains embeddings et l'entraînement de certains modèles extrêmement longs, nous empêchant de réitérer beaucoup de fois les expériences.

Enfin, nous avons dû mobiliser nos propres connaissances et effectuer beaucoup de recherches par nous-mêmes, car les cours ne mentionnent que très peu le traitement du texte, les embeddings et le topic modeling. Cependant, cette démarche nous a permis d'élargir nos connaissances, et de développer notre capacité d'adaptation face à un projet dont nous ne maîtrisons pas encore tous les enjeux techniques.

Si l'on devait continuer le projet, nous avons plusieurs pistes d'amélioration. Tout d'abord, comme mentionné ci-dessus, il serait intéressant d'annoter une centaine d'avis manuellement, afin de créer un dataset de test fiable dès le début. Nous pourrions également revoir nos trois classes, et élargir leur nombre ou les modifier en fonction des données. Nous pourrions également mieux traiter le déséquilibre des classes, qui est l'une des raisons des mauvais résultats obtenus sur la classe "service client". Le dataset étant déjà disponible, nous pourrions également tester d'effectuer la classification sur les avis segmentés en phrases. Nous ne l'avons pas fait par manque de temps, mais les avis étant clusterisés ainsi, la tâche serait peut-être plus facile pour les modèles de classification. La recherche d'un seuil de prédiction pourrait également être approfondie. A l'exception de la régression logistique, nous n'avons pas non plus testé les résultats obtenus avec les regex sur les autres modèles de classification. Enfin, nous pourrions combiner le topic modeling avec cette technique, avant de passer à la classification.